StockShark Web Application Documentation

Executive Summary

StockShark is a comprehensive stock analysis application designed to provide real-time and historical stock data analysis, enabling users to make informed investment decisions.



Leveraging Yahoo Finance API for data retrieval and MySQL for persistent storage, StockShark aims to offer seamless access to offer a robust set of features that cater to both novice and experienced investors.

Core Functionalities: Overview

1. **Real-Time and Historical Data Retrieval:** StockShark uses the YFinance API to fetch real-time stock prices and historical data. Data is pre-processed for format consistency and accuracy before being stored in a MySQL database with entry protection mechanisms to prevent data duplication.

2. **Data Persistence**: Utilizing mysql-connector-python, StockShark ensures that all stock data are persistently stored, allowing for offline access and historical data analysis.
3. **Graphical Performance Comparison**: The application features a Java-based client interface that displays interactive charts and graphs for comparing the performance of multiple stocks. This is supported by Python backend services that manage data fetching, processing, and API handling.
4. **AI-Powered Trend Prediction**: StockShark incorporates an AI analyzer built on Python. This module will use machine learning models, to predict stock trends based on historical data. This feature aims to provide users with foresight into potential future stock movements.
5. **User Interface and Experience**: The Java client will ensure a seamless user experience with a clean and intuitive interface that includes functionalities such as a portfolio tracker, watchlist, and personalized stock alerts. The system supports user sessions, including secure login/logout functionalities.

Technical specifications:

1. **Concurrency Management**: Python's asyncio library is expected to be employed to handle concurrent API calls and database operations efficiently.
2. **Security and Compliance**: StockShark will implement robust security measures including encrypted data transmission, SQL injection prevention, and compliance with GDPR for handling user data.
3. **Notification System**: An integrated notification system alerts users about significant stock movements and system anomalies, improving user engagement and system reliability.

Future work:

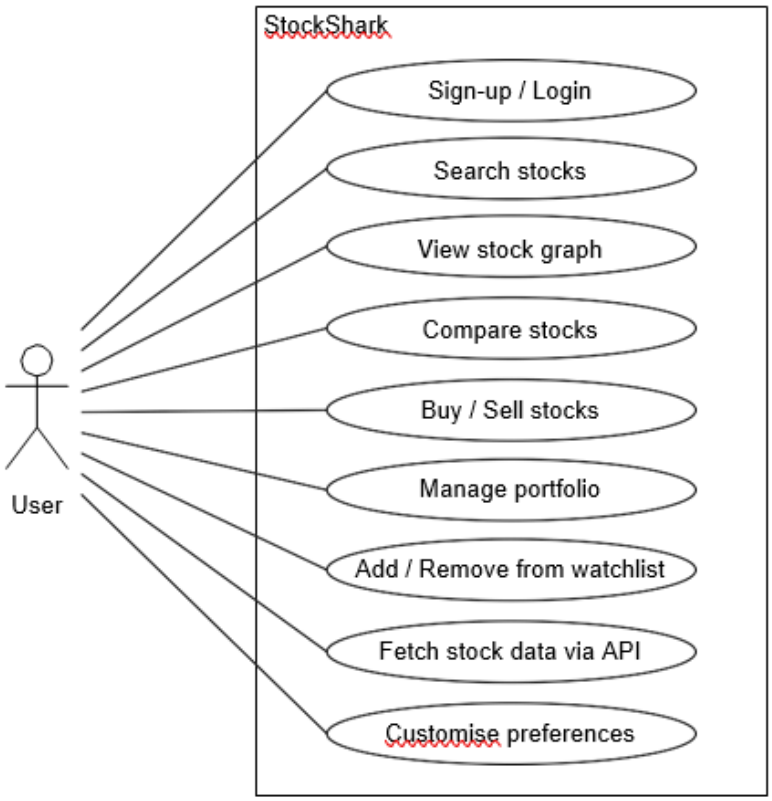
1. **Performance Optimization**: Implement caching mechanisms to improve data retrieval times and responsiveness of the application.
2. **Expanded AI Capabilities**: Future updates aim to incorporate more advanced AI-driven tools for market analysis and portfolio optimization.

Project Requirements

StockShark's core functionalities include daily stock price retrieval, persistent data storage, graphical performance comparison, and optional AI-based trend predictions. Data is fetched via Yahoo Finance API using YFinance, stored in MySQL, and visualized through Java components.

Features Collected From Yahoo Finance	**Features Collected From Google Finance**	**Features Collected From Bloomberg Terminal**
☑ **Features:** • Displays real-time and historical share prices. • Allows comparison between multiple stocks. • Provides graphical charting tools. • Offers financial news and AI-powered insights. 🔎 **Key Takeaways for Our Project:** • Implement a **simple UI** with **interactive charts**. • Support **historical data retrieval** for comparisons. • Ensure **data persistence for offline access**. • Consider **AI-based insights for trend prediction**.	☑ **Features:** • Tracks real-time market trends. • Allows personalized watchlists. • Displays stock market news and financial analysis. 🔎 **Key Takeaways for Our Project:** • Include a **stock search feature**. • Provide **side-by-side stock comparisons**. • Ensure a **clean and simple UI**.	☑ **Features:** • Professional-grade analytics and market data. • Offers advanced financial models and trend analysis. • Features **AI-powered forecasting tools**. 🔎 **Key Takeaways for Our Project:** • Consider implementing **basic AI-driven trend analysis**. • Offer **historical stock data visualization**.

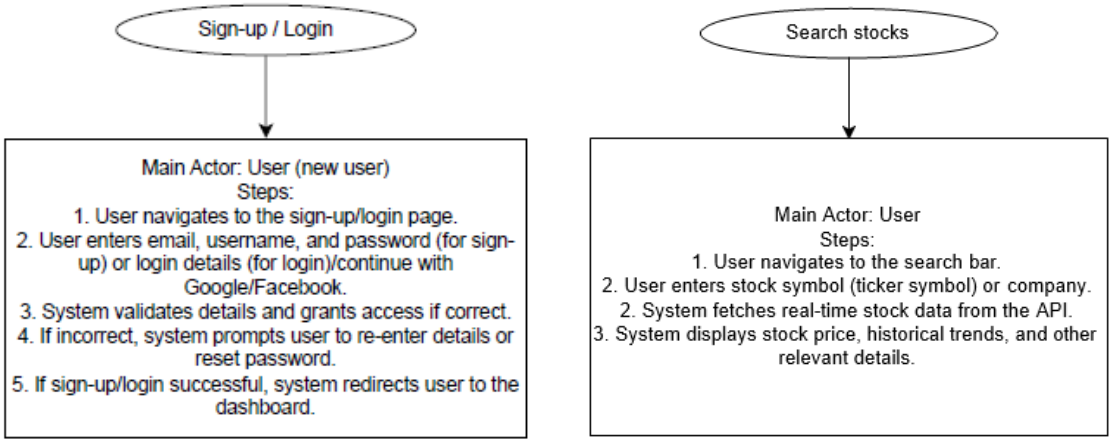
➢ Use Case Diagram

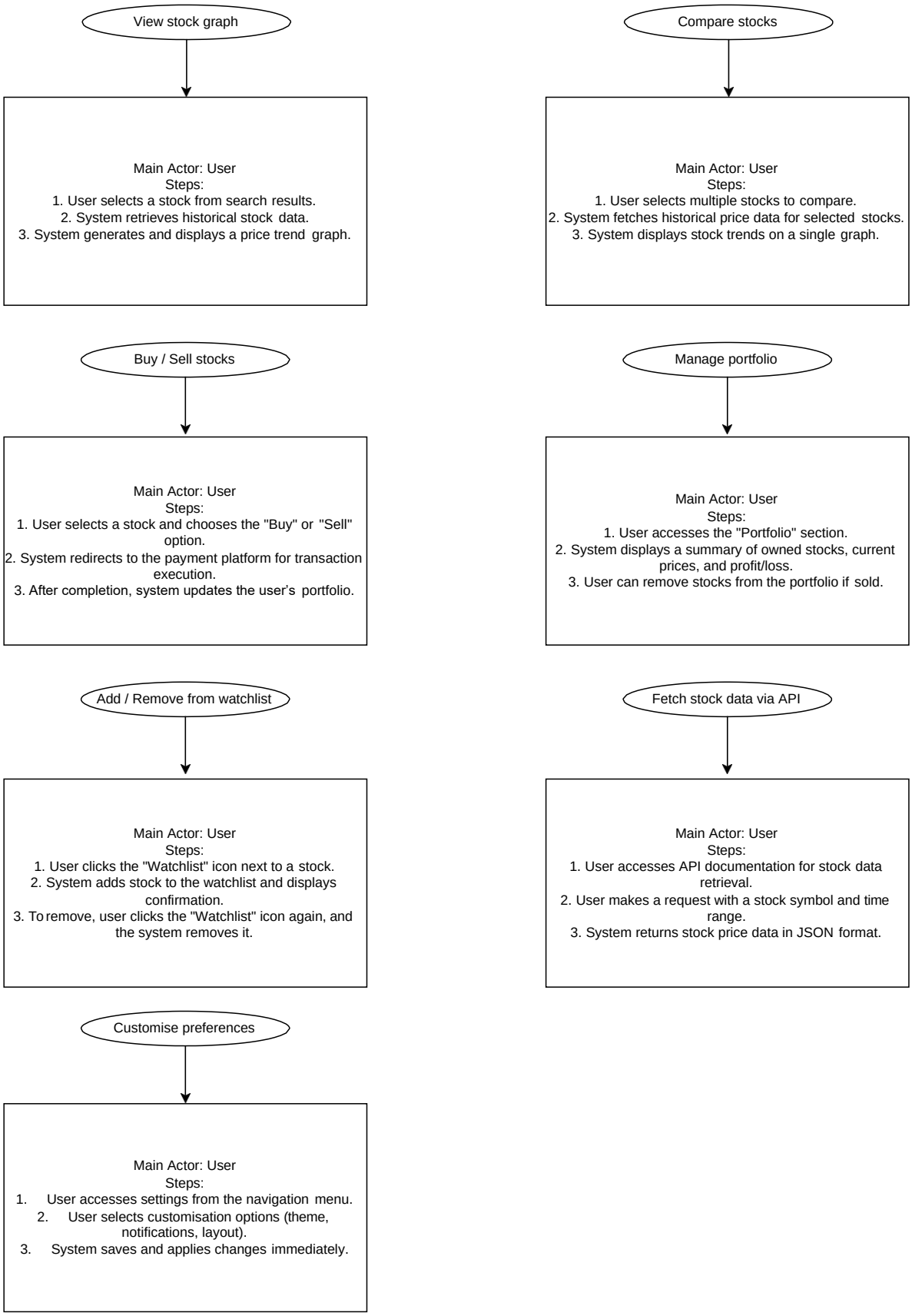


User Stories

- **Stock Price Retrieval:** As a user, I want to enter a stock symbol and date range to retrieve historical price data.
- **Data Persistence:** As a user, I want to save stock data for offline analysis.
- **Graphical Comparison:** As a user, I want to compare two stocks side by side.
- **AI Predictions:** As a user, I want AI-based trend predictions for future analysis.

➢ Scenario Details




View stock graph
Main Actor: User
Steps:
1. User selects a stock from search results.
2. System retrieves historical stock data.
3. System generates and displays a price trend graph.
Compare stocks
Main Actor: User
Steps:
1. User selects multiple stocks to compare.
2. System fetches historical price data for selected stocks.
3. System displays stock trends on a single graph.
Buy / Sell stocks
Main Actor: User
Steps:
1. User selects a stock and chooses the "Buy" or "Sell" option.
2. System redirects to the payment platform for transaction execution.
3. After completion, system updates the user's portfolio.
Manage portfolio
Main Actor: User
Steps:
1. User accesses the "Portfolio" section.
2. System displays a summary of owned stocks, current prices, and profit/loss.
3. User can remove stocks from the portfolio if sold.
Add / Remove from watchlist
Main Actor: User
Steps:
1. User clicks the "Watchlist" icon next to a stock.
2. System adds stock to the watchlist and displays confirmation.
3. To remove, user clicks the "Watchlist" icon again, and the system removes it.
Fetch stock data via API
Main Actor: User
Steps:
1. User accesses API documentation for stock data retrieval.
2. User makes a request with a stock symbol and time range.
3. System returns stock price data in JSON format.
Customise preferences
Main Actor: User
Steps:
1. User accesses settings from the navigation menu.
2. User selects customisation options (theme, notifications, layout).
3. System saves and applies changes immediately.

Architectural Design

StockShark incorporates a blend of architectural paradigms tailored to optimize maintainability, modularity and a clear separation of concerns. This section details each architectural choice and its application within the StockShark system.

Simple Architecture

Application: Simple Architecture is utilized in StockShark for straightforward and non-complex system components where high scalability and complex transactions are not primary concerns. This architecture is predominantly applied to user session management and authentication processes, such as the **Login/Logout** functionality. This choice ensures that these components are easy to implement and maintain, without the overhead of more complex architectural frameworks.

Clean Architecture

Application: The principles of Clean Architecture are foundational to StockShark, particularly in ensuring that all system dependencies are directed inwards. This approach is manifest in:

- **Core Entities** such as **User** and **Portfolio**, which lie at the heart of the system.
- **Use Cases/Business Rules Layer** with components like the **Stock Comparison** and **Notification System**, which encapsulate specific business logic independent of external interfaces.
- **Interface Adapters** such as the **Data Aggregator**, which act as intermediaries translating data between the database, external APIs, and internal business rules.

This layered approach not only supports modularity and independence in development and testing but also ensures that the system remains adaptable to changes in external APIs or database technologies.

Service-Oriented Architecture (SOA)

Application: StockShark leverages SOA to decompose the application into discrete services that can be independently developed, deployed, and maintained. Key components such as the **AI Analyzer**, **Data Aggregator**, and **Database** are developed as separate services, each responsible for distinct functionalities within the system. SOA facilitates the flexible integration of these services, promotes reusability, and enhances the system's scalability by allowing services to be scaled independently based on demand.

Model-View-ViewModel (MVVM)

Application: Within the UI layer, StockShark employs the MVVM architectural pattern, particularly effective in the **Graphical Component Display/View**. This pattern supports robust data binding between the UI elements and the underlying data models, simplifying the management of dynamic content such as real-time stock data visualization and user-specific configurations. MVVM aids in separating the presentation logic from business logic, thereby facilitating a more manageable and testable UI architecture.

Model-View-Controller (MVC)

Application: MVC is used to structure the interaction and data flow between the **Stock Comparison Graphs/Dashboard** and the underlying models. In this setup:

- **Models** manage the data and logic of the application.
- **Views** are responsible for outputting representation to the user.
- **Controllers** handle input and convert it to commands for the model or view.

This separation helps manage complex data interactions and user interfaces, making the system more understandable and easier to debug.

Component and Use Case Diagrams

Component diagrams illustrate the structure of the system by showing its components and their relationships, which is crucial for understanding how different parts of StockShark interact, such as data retrieval, storage, and user interfaces. Use case diagrams map user interactions with the system, helping define user requirements and system functionality.

